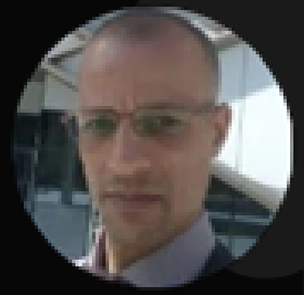
Higher School of Computer Science
08 May 1945 of Sidi Bel Abbes

Software Engineering For Data Science (SEDS)

Getting Started with Python & Jupyter Notebook

PRESENTED BY

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



PRESENTED ON 11-10-2026

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Python

What is Python?

- **Python is a programming language**
- **Python is an interpreted language:** lines of code are executed one at a time.
- **Python has two active versions:**
  - Python 2.x
  - Python 3.x

(**caution:** slight differences between versions!)

Why Python?

- Widely used programming language by **Data Scientists**.
- **Multi-purpose language**
- Great **data science** libraries (Extensible ecosystem)
- Easy to learn / use

https://www.python.org/

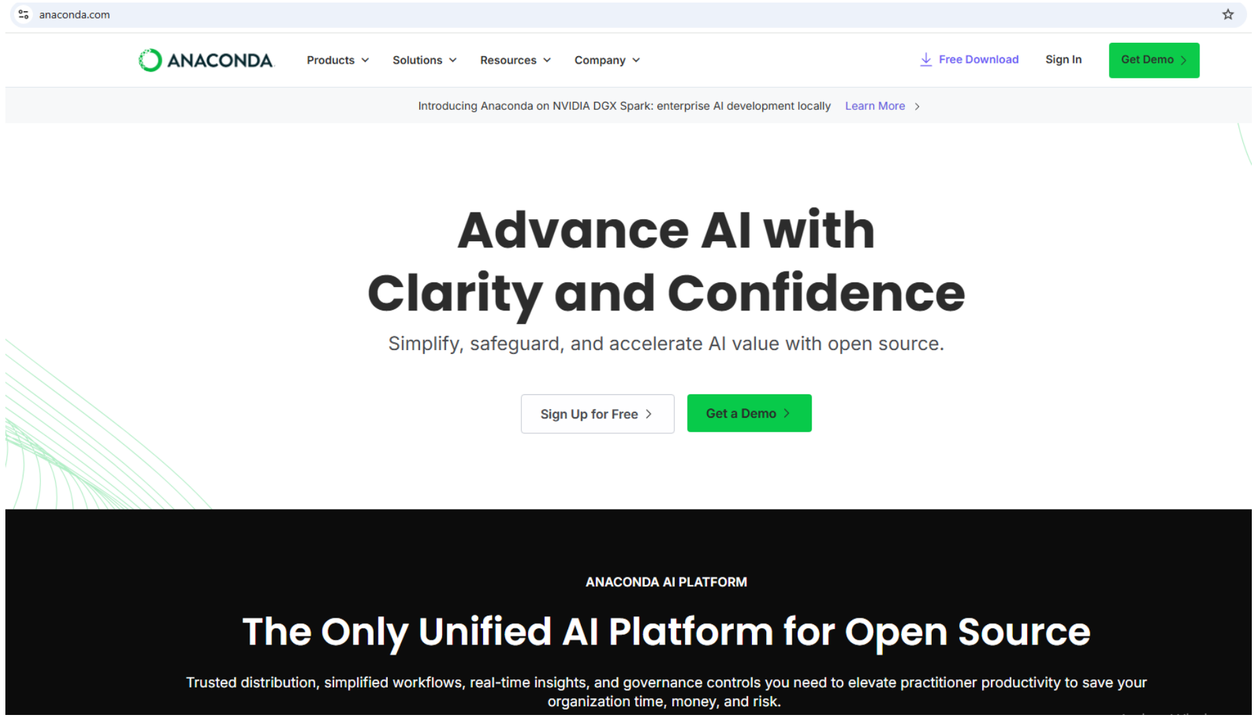
DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Python

Installation and Setup:

- Use **Anaconda distribution** provided by **Continuum Analytics**.
  - The easiest way to perform **Python/R data science** and **machine learning** on a single machine.
  - Comes with **conda**, **Python**, and over **150 scientific packages** and their dependencies.
  - Offered in both **Python 2.7** and **3.x**.



https://www.anaconda.com/

Note: Your system may come shipped with a default Python interpreter. Even so, we strongly encourage to follow the steps upcoming next in order to:

1. replicate the exact environment used in this class and
2. practice with possible installation/configuration issues.

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Python

Installation and Setup:

Distribution

Register to get everything you need to get started on your workstation including Cloud Notebooks, Navigator, AI Assistant, Learning and more.

- Easily search and install thousands of data science, machine learning, and AI packages
- Manage packages and environments from a desktop application or work from the command line
- Deploy across hardware and software platforms
- Distribution installation on Windows, MacOS, or Linux

For students and researchers, create an account for free using your education domain. Learn more about Anaconda for Education plan.

Download Now

Get access in 30 seconds. Completely free.*

Get Started >

Returning Users >

*Subject to our Terms of Service. Use of Anaconda's offerings at an organization of more than 200 employees/contractors requires a paid business license unless your organization is eligible for discounted or free use. See Pricing.

https://www.anaconda.com/

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Python

Running a Python Code:

- Using the REPL (Read-Eval-Print-Loop) (results in real time, type code line-by-line) with:
  - Python Shell or,
  - IPython Shell.
  - Put your code in a file (e.g. test.py), then execute python test.py in a terminal to run the file line-by-line.

```
Users > macbook > test.py
  1   print('Hi, Data Scientists')
```

```
macbook — -bash — 80×24
(base) MacBook-Pro-de-Macbook:~ macbook$ python test.py
Hi, Data Scientists
(base) MacBook-Pro-de-Macbook:~ macbook$
```

The Python Shell

```
macbook — jupyter_mac.command — python — 80×24
(base) MacBook-Pro-de-Macbook:~ macbook$ python
Python 3.8.5 (default, Sep  4 2020, 02:22:02)
[Clang 10.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 3*8
24
>>> print('Hello Data Scientists')
Hello Data Scientists
>>>
```

The IPython shell

```
macbook — IPython: Users/macbook — ipython — 80×24
(base) MacBook-Pro-de-Macbook:~ macbook$ ipython
Python 3.8.5 (default, Sep  4 2020, 02:22:02)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.19.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: 3*8
Out[1]: 24

In [2]: print('Hi, Data Scientists')
Hi, Data Scientists

In [3]:
```

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Python

Installing Python Packages and Managing virtual environments:

- **Packages** extend python functionalities and make certain tasks easier, such as building and running machine learning models.
  - **Numpy**: For Matrices, and N-Dim Arrays, and more.
  - **Scipy**: For algorithms optimization, integration, interpolation, eigenvalue problems, algebraic equations, differential equations, statistics and many.
  - **Pandas**: For Data analysis and manipulation.
  - **Scikit-learn**: For Machine learning and predictive data analysis
  - **TensorFlow** and **PyTorch**: End-to-end advanced deep learning platform ecosystems.

NumPy

pandas

SciPy

scikit learn

TensorFlow

PyTorch

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Python

Installing Python Packages and Managing virtual environments:

Two primary ways to install and manage packages – **conda** and **pip:**

Pip:

- The package installer for **Python**
- The classic way to manage **packages** in **Python**.
- pip has no **dependency resolution**!
- General Pip commands:

```
pip install <PACKAGENAME>
pip install --upgrade <PACKAGENAME>
pip uninstall <PACKAGENAME>
pip freeze                  # Prints all versions
```

Conda:

- The **package manager** that comes with the **Anaconda** Python distribution
- Conda has **dependency resolution**
  - **Difficult** when using pip.
- General Conda commands for packages:

```
conda install <PACKAGENAME>
conda update <PACKAGENAME>
conda remove <PACKAGENAME>
conda list                  # list all packaged
```

Upgrading a conda package with pip (or vice versa) will break stuff!

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Python

Installing Python Packages and Managing virtual environments:

- **Virtual environments** ⇒ Separate **Python installation** to prevent overlap with the **base** Python installation.
- Useful while:
  - Using **different** versions of **Python** (e.g., **3.9** and **3.8**)
  - Having **different** virtual environments with certain packages for different projects.
- Gain more reproducibility.
- Many ways in which to run virtual environments with Python: **virtualenv**, **venv**, **pipenv**, **conda**, and more.

Some useful Conda env. Commands

```
#Creating env.
conda create --name <ENVNAME>
conda create -n <ENVNAME>
conda create --clone <ENVNAME> --name <ENVNAME>
conda env create --file <FILENAME.txt>
conda create -n <ENVNAME> <PACKAGENAME>=<VERS>
#Activating or Deactivating an env.
#Windows:
activate
deactivate
#Linux and macOS:
conda activate
conda deactivate
#export environments for sharing or moving it
conda env export --from-history > <FILENAME.yml>
```

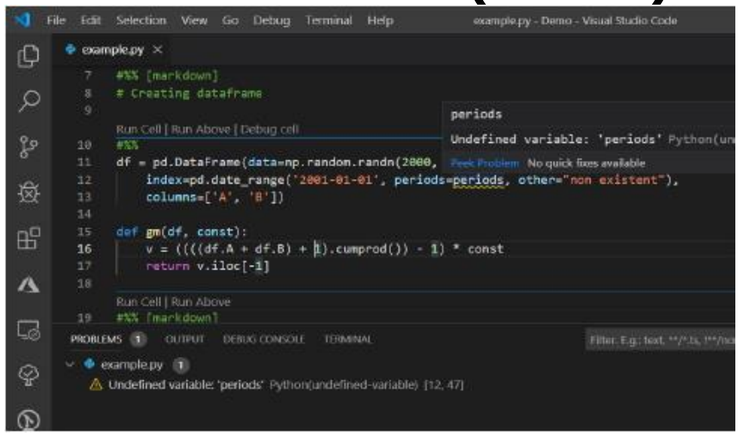
DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



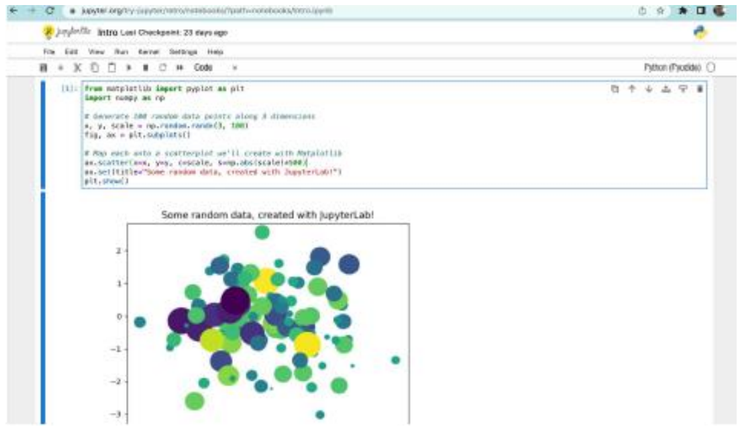
Introduction to Python

Integrated Development Environments (IDEs):

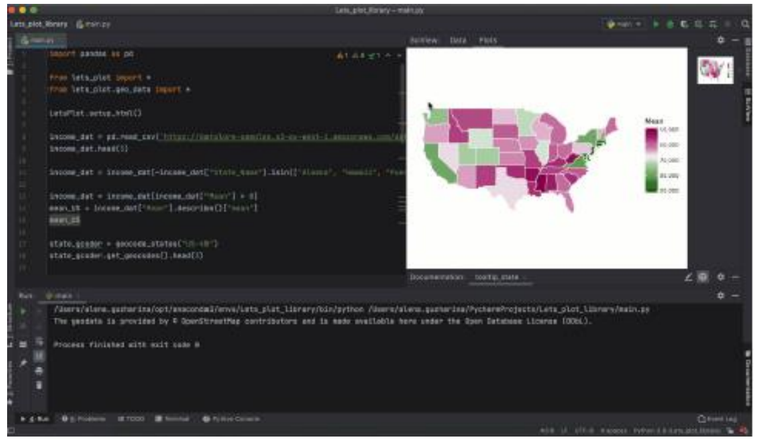
- IDEs provide productivity tools such as syntax highlighting, auto-completion, debugging, and more rich features.
  - Jupyter Notebook from Jupyter.
  - VS Code from Windows
  - PyCharm from JetBrains
  - Spyder, an IDE currently shipped with Anaconda.



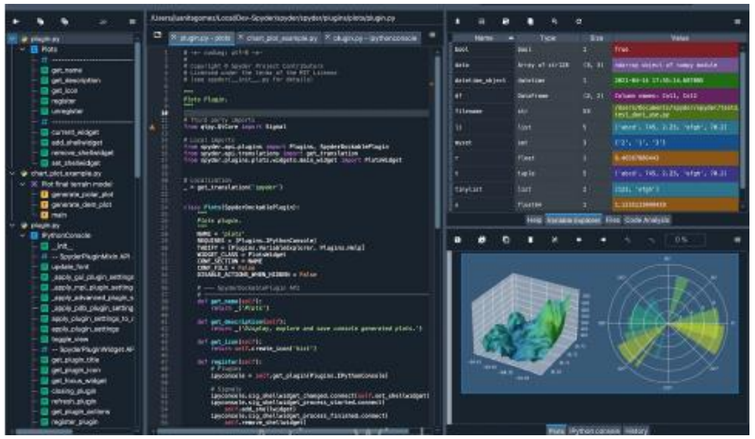
VS Code



Jupyter Notebook



PyCharm



Spyder

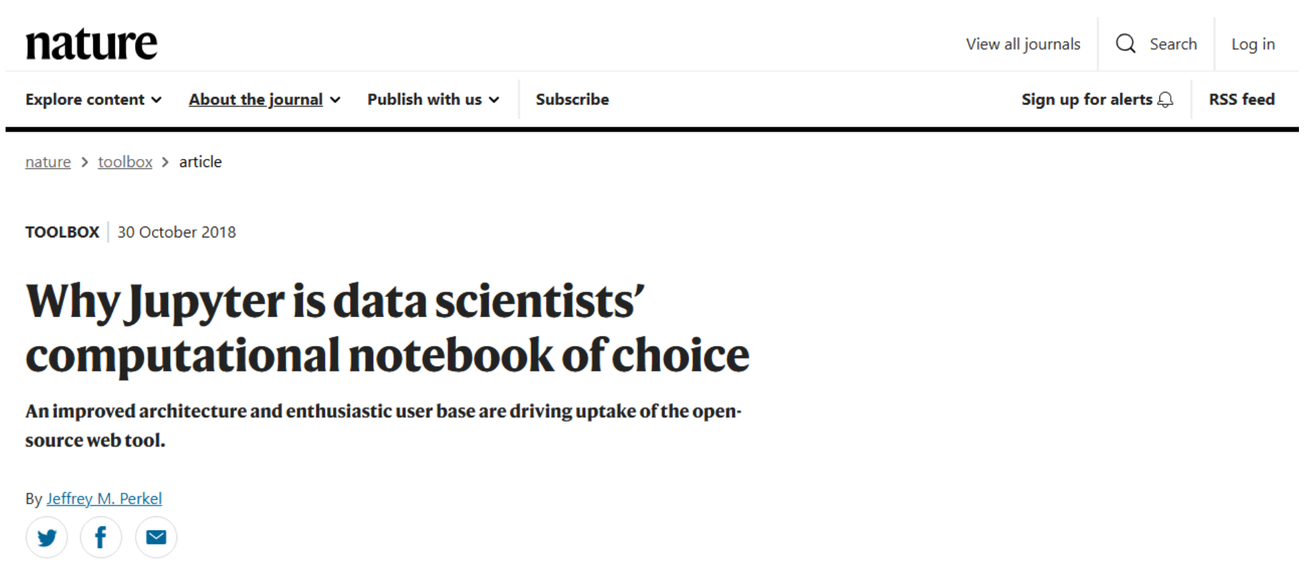
DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

Jupyter Notebooks: What?

- A **Jupyter notebook** (formerly Ipython Notebook) is a web-based interactive computational environment for creating notebook documents.
- It makes scientific programming **interactive**!
- It is among the **top data scientists' computational notebook** of choice.



nature

View all journals | Search | Log in

Explore content | About the journal | Publish with us | Subscribe | Sign up for alerts | RSS feed

nature > toolbox > article

TOOLBOX | 30 October 2018

Why Jupyter is data scientists' computational notebook of choice

An improved architecture and enthusiastic user base are driving uptake of the open-source web tool.

By Jeffrey M. Perkel

Perkel, Jeffrey M. "Why Jupyter is data scientists' computational notebook of choice." *Nature* 563, no. 7732 (2018): 145-147.

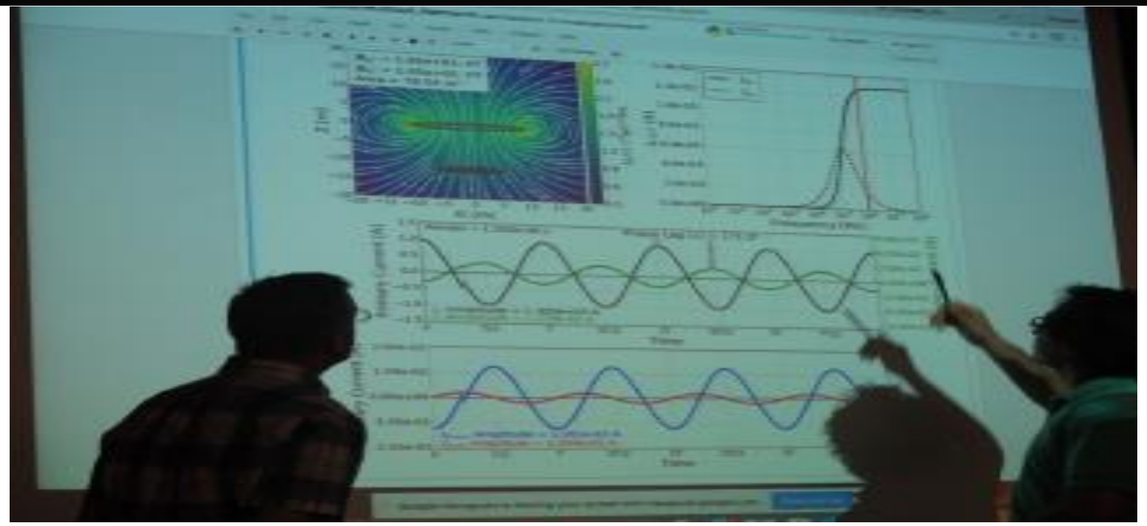
DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

Jupyter Notebooks: What?

- **Jupyter Notebooks** are documents that allow you to bring together **data**, **code**, and **prose**, to tell an **interactive**, computational **story** (.. *All in one place*).
- More **engaging** with **practitioners**.
- Adoption of literate programming by embracing standard web technologies (i.e., mostly **HTML5**, **JavaScript**, and **CSS**)





DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



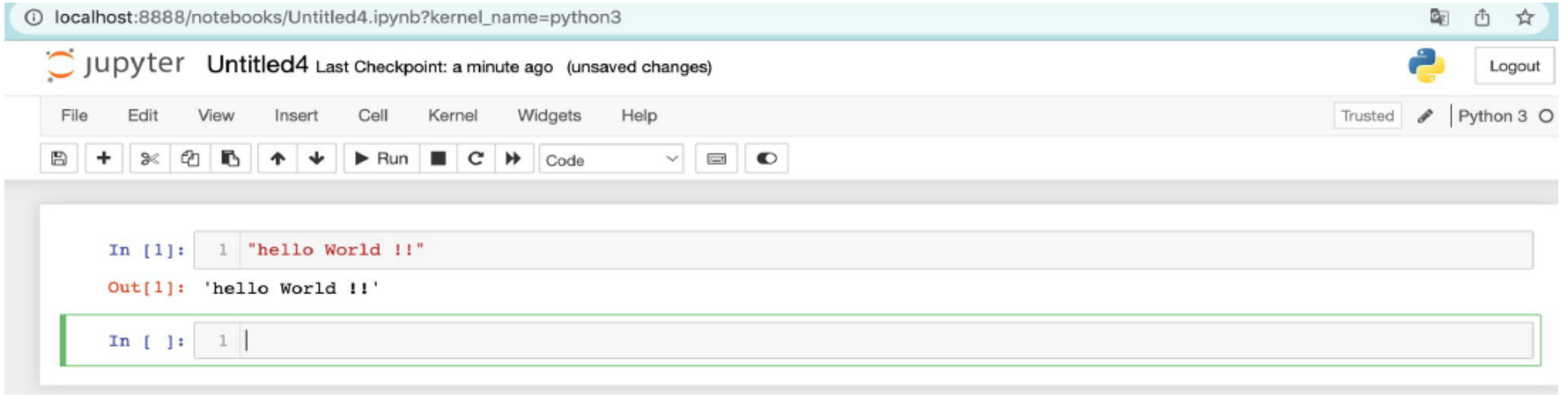
Introduction to Jupyter Notebooks

Jupyter Notebook vs Jupyter Lab

- Both equally functional and compatible (but **Lab** is the *Next-Generation Notebook Interface*)

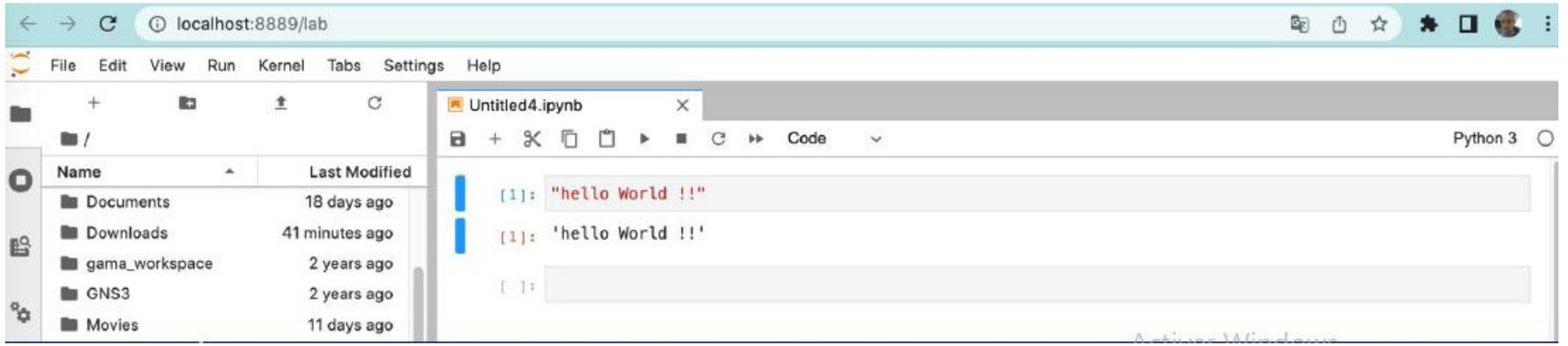
Notebook

More Suitable for beginners



JupyterLab

More Suitable for Advanced users



DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

Jupyter Notebook's Overall Architecture

- **Jupyter Notebook** ⇒ IPython kernel (**backend**) + the web-based interface provided with Notebook (**frontend**).
  - ⇒ Stores *code* and *output* in an editable document called a **notebook**.
  - ⇒ Saved on disk as a **JSON** file with a **.ipynb** extension.
  - Communication handlled by **ZeroMQ**.
- **Jupyter Notebook** on a local machine without Internet access:
  - Via a web browser running on the same machine where the **Notebook server** is running,
  - Via **a remote server** and accessed through the Internet.

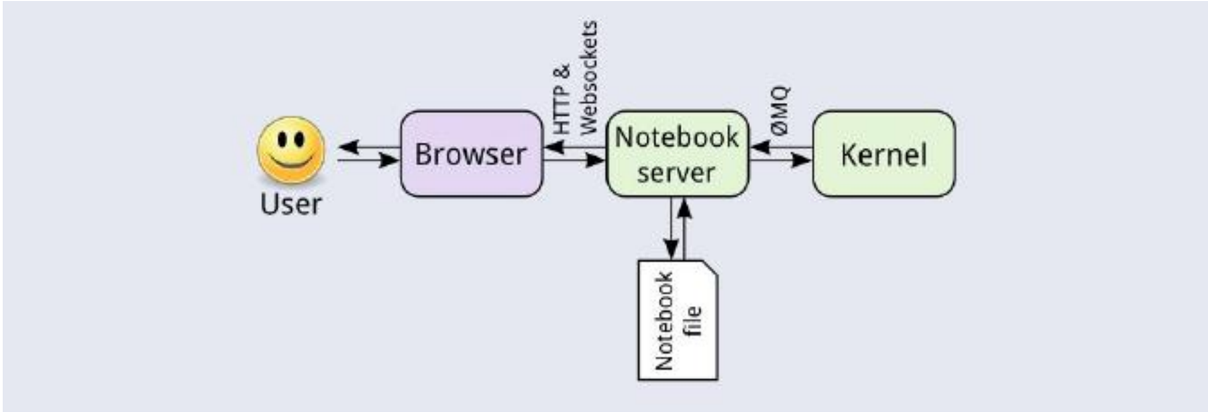


Fig. High-level architecture of Jupyter Notebook

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

Installation and Setup:

- **Prerequisite:** Python Interpreter
  - Python is a requirement for installing Jupyter Notebook.
  - Python and Jupyter Notebook can be installed separately.

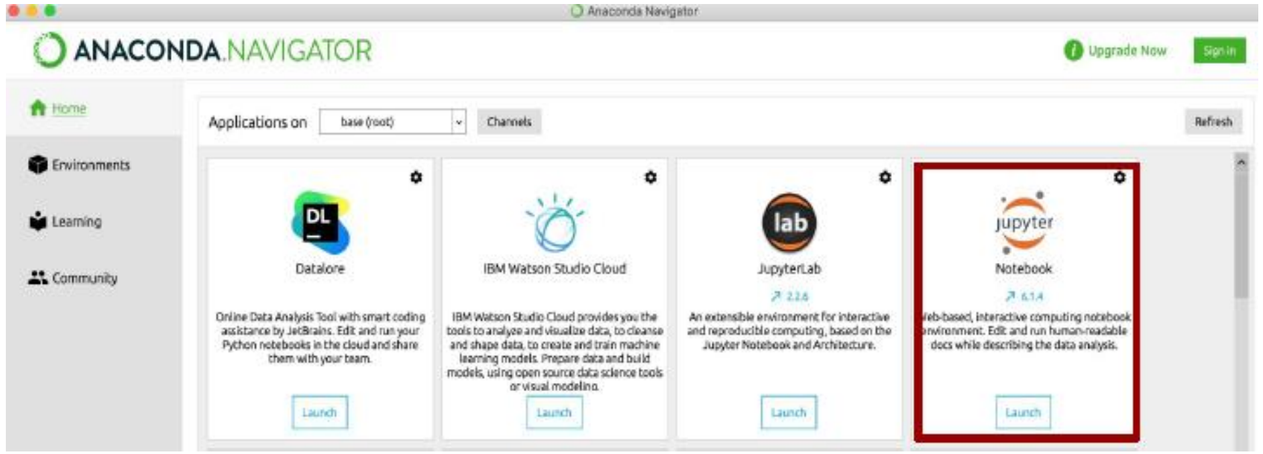
- **Installation:**
  - Using Anaconda Python Distribution (highly recommended)
  - Using pip (only for skilled users).

Using Anaconda:

- Open a shell and type:

```
jupyter notebook
```

- Or launch it using the Anaconda Navigator



DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

Installation and Setup:

- **Prerequisite:** Python Interpreter
  - Python is a requirement for installing Jupyter Notebook.
  - Python and Jupyter Notebook can be installed separately.

- **Installation:**
  - Using Anaconda Python Distribution (highly recommended)
  - ➡ Using pip (only for skilled users).

Using pip:

- Better for experienced and skilled Python users.
- Make sure you have the latest pip:

```
pip3 install upgrade pip
```

- Install Jupyter Notebook as follows:

```
pip3 install jupyter
```

- Run the Jupyter Notebook application in a shell command:

```
jupyter notebook
```

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

Jupyter Notebook UI Components:

- Two main types of cells: Markdown cells and Code cells.
  - **A Markdown cell:** Rich text with classic formatting options: (e.g.: bold or italics), + advanced features (e,g.: links, images, HTML elements, LaTeX mathematical equations, and much more.
  - **A Code cell:** Code to be executed by the kernel.

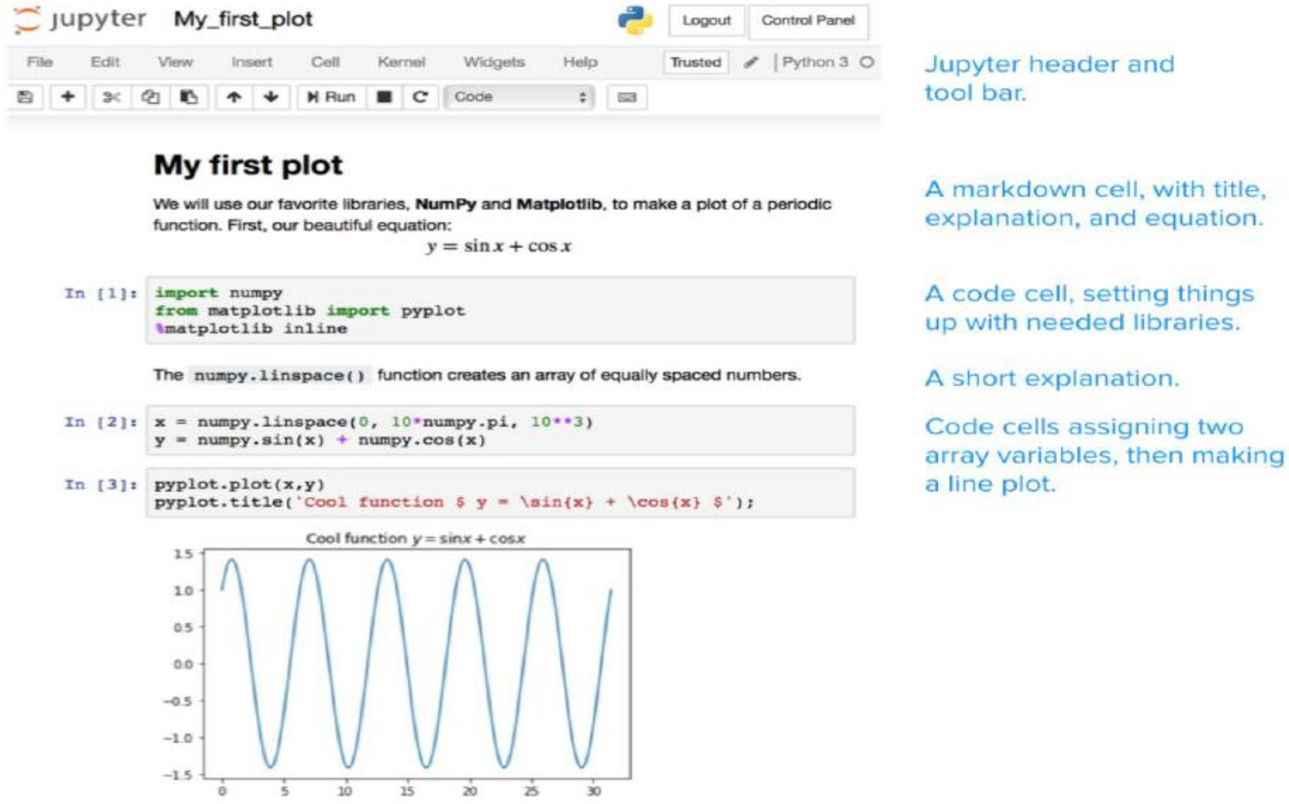


Fig. An Example of a Jupyter Notebook Document

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

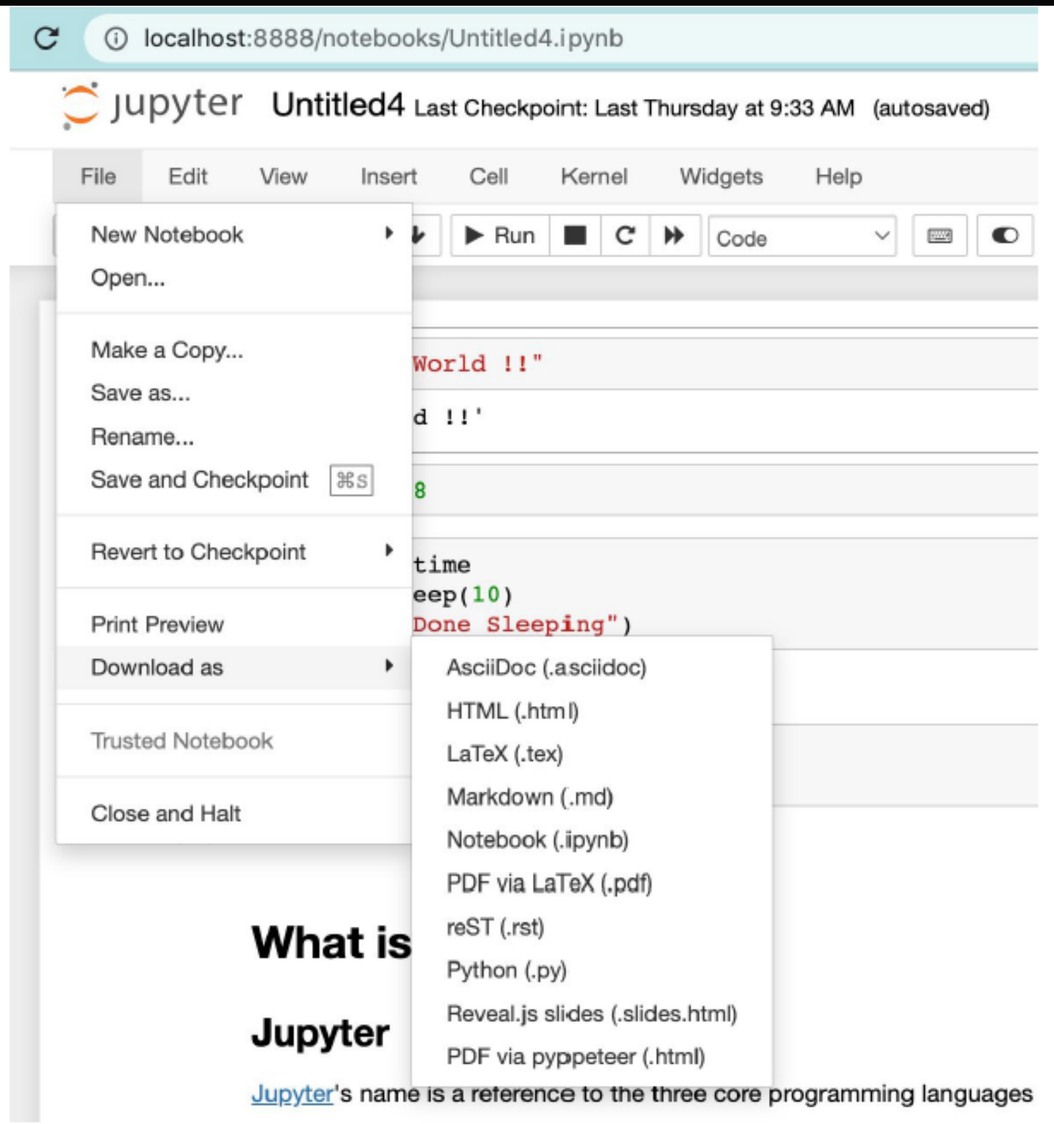
Advanced Features:

Share Jupyter Notebooks

- **Notebooks:** documents saved as JSON files.
- We don't want to share our **notebooks** with others as JSON files.

⬇

- Jupyter allows to download your notebook as:
  - Notebook (**.ipynb**),
  - Python script (**.py**),
  - HTML (**.html**),
  - Markdown (**.md**),
  - reStructuredText (**.rst**),
  - LaTeX (**.tex**)
  - PDF (**.pdf**) file, ...



DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz

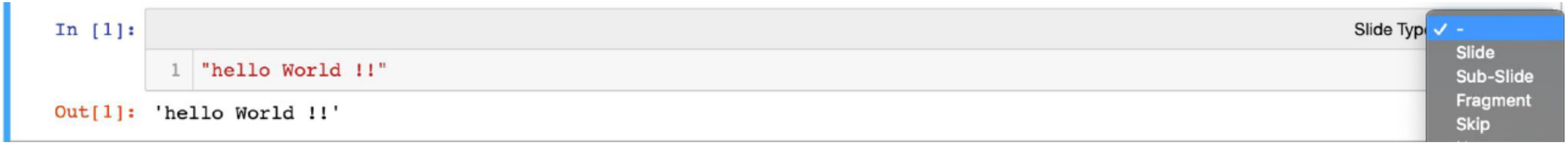


Introduction to Jupyter Notebooks

Advanced Features:

Converting Jupyter Notebook to Slides

- To convert a notebook into a **reveal.js** presentation, go to the **View menu** and set **Cell Toolbar** to **Slideshow**.
- Organize the cells of your notebook into **slides** and **sub-slides**.



- Convert it to the slide format using **nbconvert** as follows:

```
jupyter nbconvert {NOTEBOOK_NAME}.ipynb to slide --post serve
```

  - This will generate a **{NOTEBOOK_NAME}.slides.html** file in the same directory of your notebook file
  - You can open it on a new tab of your system default browser at

```
http://localhost:8000/{NOTEBOOK_NAME}.slides.html/
```

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



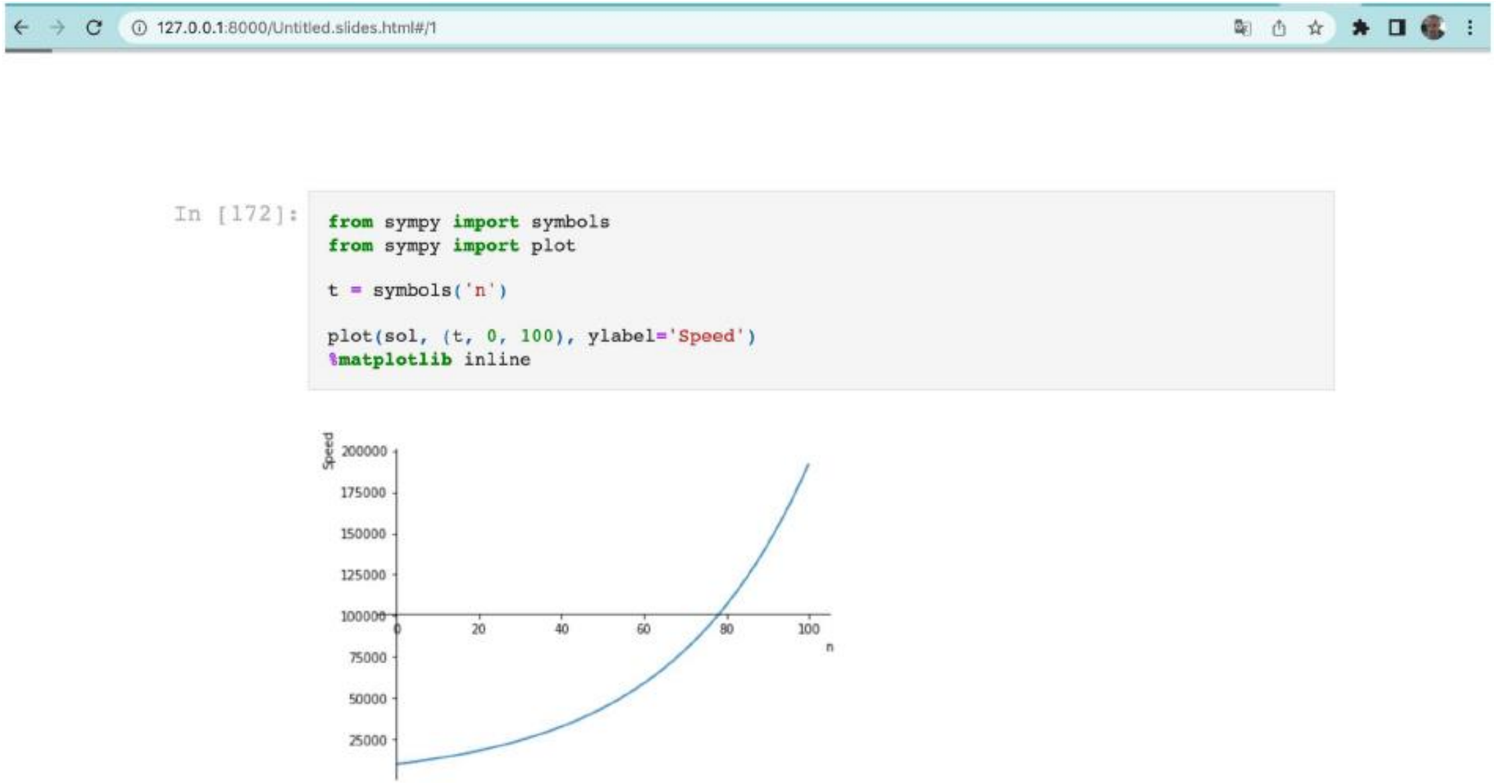
Introduction to Jupyter Notebooks

Advanced Features:

Converting Jupyter Notebook to Slides

- Output Result Example ➡

- You may apply templates from github projects to customize and build attractive slides.



DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Introduction to Jupyter Notebooks

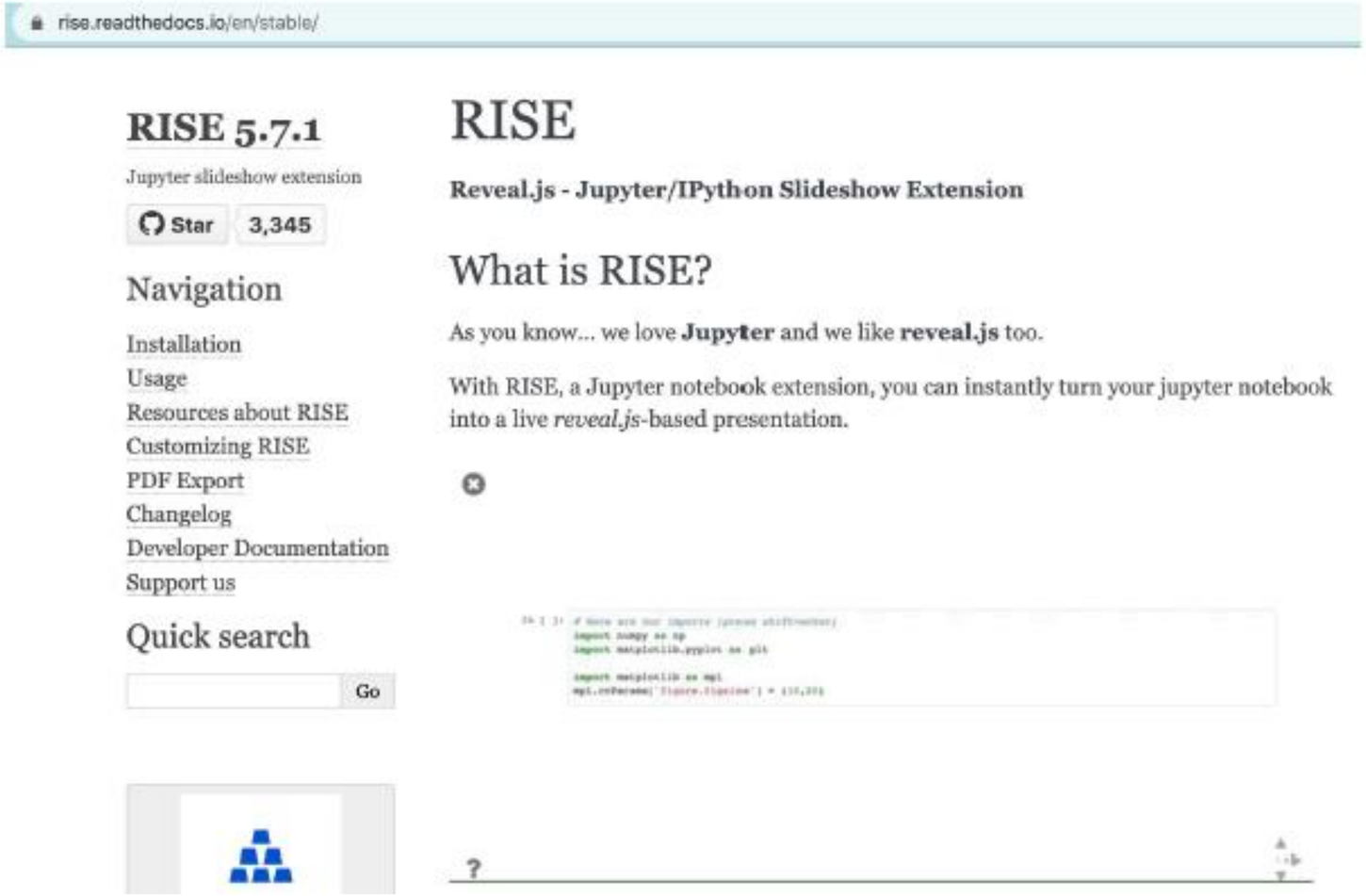
Advanced Features:

Converting Jupyter Notebook to Slides

- An alternative way with RISE Plugin:
- Install RISE plugin as a Jupyter Notebook
- extension as follows:

```
conda install -c conda-forge rise
```

```
pip install rise
```



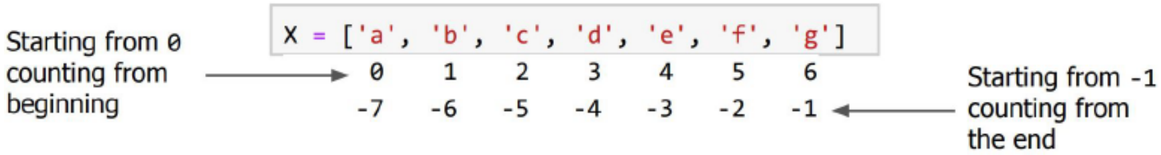
DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Python Basics

Basic Data Types:

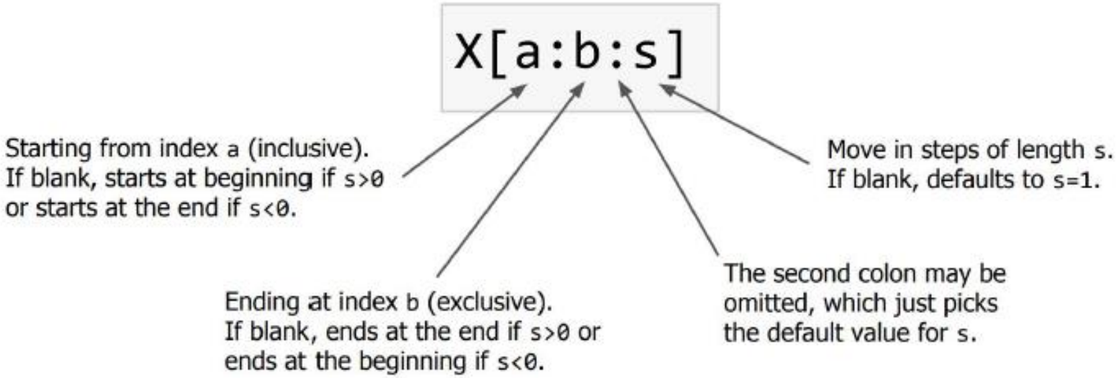
- Compound Data Types: (list, tuple) Combine basic data types together
  - Indexing: Each item can be referenced from start or end.




```
print(X[5] + X[-7] + X[-5] + X[4])
```

```
'face'
```

  - Slicing: Can specify start, end, and step length to access a range of elements.




```
print(X[1:4])        #['b','c','d']
print(X[3:])         #['d','e','f','g']
print(X[-5:5])       #['c','d','e']
print(X[0:4:2])      #['a','c']
print(X[:3])         #['a','b','c']
print(X[3::-1])      #['d','c','b','a']
```

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Python Basics

Basic Data Types:

- **Compound Data Types:** (dict) Combine basic data types together
  - Dictionaries (dict) are similar to lists, except they are indexed using strings (keys) rather than integers.
  - dict can be seen as a set of key:value pairs, with the requirement that the keys are unique.

```
Ages={Ali:19, 'Ahmed':22}
ages['Ahmed']
```

```
22
```

```
scores={'Ali':[75,86,92],'Ahmed':[71,95,84]}
scores['Ahmed']
```

```
[71,95,84]
```

```
db={'Ali': {'high school': 'Ali Mellah',
            'college': 'ESI-SBA',
            'age': 19,
            'scores': [75,86,92]
            },
    'Ahmed':{'high school': 'El Idrissi',
            'college': 'ESI-ALG',
            'age': 22,
            'scores': [71,95,84]
             }
    }
db['Ali']['college']
```

```
ESI-SBA
```

- Iterating over a dict:

```
for key, value in db.items():
    print(f'{key}:{value}')
```

```
Ali:{'high_school': 'Ali Mellah', 'college': 'ESI-SBA', 'age': 19, 'scores': [75, 86, 92]}
Ahmed:{'high_school': 'El Idrissi', 'college': 'ESI-ALG', 'age': 22, 'scores': [71, 95, 84]}
```

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Python Basics

Basic Data Types:

- **Compound Data Types:** (dict) Combine basic data types together
  - **Tabular Data Example:**
    - **Each row**: one observation
    - **Each column**: features of each observation.

City	Area in (Km^2)	Population in (Millions)
London	1,572.00	8.61
Berlin	891.70	3.56
Madrid	604.31	3.16
Rome	1,285.00	2.87
Paris	105.40	2.27
...	...	...

- Many Possible Representations
  - Row-first:

```
data = {'London':{'Area':1572,
                  'Population':8.61
                 },
        'Berlin': {'Area':891.70,
                   'Population':3.56
                  },
        ...
        }
```

  - Column-first:

```
data = { 'Area': { 'London':1572,
                   'Berlin':891.70,
                      ...
                    }
         'Population': { 'London':8.61,
                         'Berlin':3.56,
                         ...
                         }
        }
```

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Python Basics

Basic Data Types:

- **Compound Data Types:** (dict) Combine basic data types together
  - **Tabular Data Example:**
    - **Using Pandas DataFrame:**
      - **Pandas** ⇒ a **Python package** providing fast, flexible, and expressive data structures designed to make working with "**relational**" or "**labeled**" data both easy and intuitive.
      - **Pandas** ⇒ the fundamental high-level building block for doing practical, real world data analysis in **Python**.

```python
import pandas as pd

data = {'London':{'Area':1572,
                  'Population':8.61
                 },
        'Berlin': {'Area':891.70,
                   'Population':3.56
                  },
        'Madrid': {'Area':604.31,
                   'Population':3.16
                  },
        'Rome': {'Area':1285.00,
                 'Population':2.87
                  },
        'Paris': {'Area':105.40,
                   'Population':105.40
                  }
        }

df=pd.DataFrame.from_dict(data,orient='columns')
df
```

	London	Berlin	Madrid	Rome	Paris
Area	1572.00	891.70	604.31	1285.00	105.4
Population	8.61	3.56	3.16	2.87	105.4

```python
import pandas as pd

data = {'London':{'Area':1572,
                  'Population':8.61
                 },
        'Berlin': {'Area':891.70,
                   'Population':3.56
                  },
        'Madrid': {'Area':604.31,
                   'Population':3.16
                  },
        'Rome': {'Area':1285.00,
                 'Population':2.87
                  },
        'Paris': {'Area':105.40,
                   'Population':105.40
                  }
        }

df=pd.DataFrame.from_dict(data,orient='index')
df
```

	Area	Population
London	1572.00	8.61
Berlin	891.70	3.56
Madrid	604.31	3.16
Rome	1285.00	2.87
Paris	105.40	105.40

Note: More Details will be available in the upcoming Chapters

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Python Basics

Functions:

- **Built-In:** Part of Python packages/libraries.

```
from math import sqrt
print("Hello World!")
sqrt(91)
```

- **User-Defined:** Written by the user to achieve a specific purpose.

```
def density(area,population):
    dens = population/area
    return dens
```

- **Positional vs Named arguments:** Specify a default value to make an argument optional.

```
def weather( day = 'Monday', forecast = 'cloudy' ):
    print('It will be', forecast, 'on', day)
```

- **Arguments** can be specified **positionally** or by name.

```
#It will be cloudy on Monday
weather()
#It will be sunny on Tuesday
weather('Tuesday','sunny')
#It will be rainy on Monday
weather(forecast='rainy')
#It will be dry on Friday
weather(forecast='dry', day='Friday')
```

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz



Python Basics

Packages:

- **Packages** (**Libraries**) ⇒ makes Python so powerful for data science.
  - **Built-in Packages**: Provide core functionality.
  - **Community Packages (PyPI/GitHub)**: Deliver powerful, specialized tools that extend Python's capabilities..
  - A **package** can be:
    - A directory of modules (e.g., **numpy**, **pandas**)
    - A single module (e.g., **time**, **math**)
    - A hierarchy containing sub-packages.

- Import all functionalities from Packages.

```
import numpy as np
a=np.array([1,2,3])
```

- Import a specific function from a package.

```
from math import sqrt, sin
b=sin(45)*sqrt(30)
```

- Import a Sub-Package.

```
import urllib.request
urllib.request.urlopen('https://www.pypi.org')
```

⬇

- Best way to shorten the long call.

```
from urllib.request import urlopen
urlopen('https://www.pypi.org')
```

Higher School of Computer Science
08 May 1945 of Sidi Bel Abbes

Thank You For Your Attention

DR. B. KHALDI
Assoc. Prof.
email: b.khaldi@esi-sba.dz

